

INTRODUCCIÓN AL LENGUAJE JAVASCRIPT

1. CARACTERÍSTICAS DE JAVASCRIPT

JavaScript es un lenguaje de programación interpretado (un lenguaje de programación para el que la mayoría de sus implementaciones ejecuta las instrucciones directamente, sin una previa compilación del programa) que se utiliza **fundamentalmente** para dotar de comportamiento **dinámico a las páginas web**. Por ello, **cualquier navegador web actual** incorpora un intérprete para código JavaScript.

Cualquier navegador web actual incorpora un intérprete para código JavaScript.

Algunas características de JavaScript:

- La sintaxis de JavaScript **se asemeja a la de C++ y la de Java**. De hecho, JavaScript está **basado** en el concepto de **objeto**, pero no es un lenguaje orientado a objetos
- Los objetos JavaScript utilizan herencia basada en prototipos. (Java o C++: lenguajes basados en clases → VS : objetos en JavaScript son "contenedores" dinámicos de propiedades que poseen un enlace a un objeto prototipo o modelo).
- Es un **lenguaje débilmente tipado**, lo que permite que una **variable** pueda contener valores de **distintos tipos en diferentes momentos de la ejecución** del programa. A cambio, muchos **errores** de programación **no aparecen hasta que el programa es ejecutado**, ya que el compilador es incapaz de detectarlos.
- Una **característica poco deseable** de JavaScript es el hecho de que, por defecto, **todas** las **variables son globales**. Es decir, aquellas variables que no son definidas dentro de otra variable se ubican en un espacio de nombres común denominado global object. En general, el uso de variables globales no es una buena práctica de programación.

2. "HOLA MUNDO" CON JAVASCRIPT

Una **ventaja** de JavaScript es que **un explorador web y un simple editor de textos** constituyen un completo **entorno de desarrollo**.

Para **ejecutar** cualquier programa JavaScript tenemos que **embeber** el código JavaScript en una **página HTML** utilizando la **etiqueta <script></script>**.

Hay **dos formas** de embeber el código JavaScript utilizando la **etiqueta <script>**:

- Incluirlo **directamente en la página HTML** mediante la etiqueta <script>. Crea un fichero **Hola_mundo1.html** y copia y pega el siguiente código en el fichero:

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="utf-8">
    <title>Hola Mundo</title>
    <script type="text/javascript">
      alert("Hola mundo en JavaScript");
    </script>
  </head>
  <body></body>
</html>
```

El atributo **type="text/javascript"** ya no es necesario en HTML5

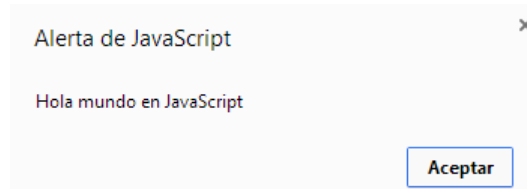
- Utilizar el atributo **src** de dicha etiqueta para **especificar el fichero que contiene el código JavaScript** (**Hola_mundo2.html**)

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="utf-8">
    <title>Hola Mundo</title>
    <script src="scripts/Hola_mundo2.js"></script>
  </head>
  <body></body>
</html>
```

El fichero `Hola_mundo2.js` debe contener esta línea de código y estará dentro de una **carpeta scripts** la cual esta en la **misma carpeta** que `Hola_mundo2.html`:
`alert('Hola mundo en JavaScript')`

Nota: El elemento meta con atributo charset en un documento HTML está el destinado a indicar la **codificación de caracteres utilizada** (charset). Si se utiliza la codificación UTF-8 (estándar en HTML5), este formato es considerado el más idóneo para web y e-mail.

Si abriésemos los dos ficheros `Hola_mundo1.html` y `Hola_mundo2.html` con un navegador web el resultado sería exactamente el mismo: el navegador muestra el mensaje "Hola mundo en JavaScript".



No obstante, la forma **recomendada** de proceder es la segunda, ya que al **localizar el código JavaScript en un fichero externo** estamos facilitando la **reutilización** y **modularización** de dicho código.

3. EL LENGUAJE JAVASCRIPT: SINTAXIS

MAYÚSCULAS Y MINÚSCULAS

El lenguaje JavaScript **distingue** entre mayúsculas y minúsculas, no es lo mismo utilizar la función `alert()` que `Alert()`.

COMENTARIOS EN EL CÓDIGO

Existen **dos formas de insertar comentarios**.

-Una de ellas es a través de la doble barra (`//`), con la cual comentamos una sola línea de código.

-La otra forma que podemos utilizar para comentar un código es a través de los signos `/*` al inicio del comentario y los signos `*/` al final del mismo. De este modo podemos comentar varias líneas de código.

```
<script>
    //Este modo permite comentar una sola línea
    /*Este modo permite realizar
    comentarios de
    varias líneas*/
</script>
```

TABULACIÓN Y SALTOS DE LÍNEA

JavaScript **ignora los espacios, las tabulaciones y los saltos de línea** presentes entre los símbolos del código.

En el momento en que los programas empiezan a ser complejos, podemos apreciar la **utilidad** de emplear las tabulaciones y los saltos de línea adecuados **para mejorar la presentación y la legibilidad del código**.

Versión 1:

```
<script>var i,j=0;
for (i=0;i<5;i++) { alert("Variable i: "+i);
for (j=0;j<5;j++) {if (i%2==0) {
document.write
(i + "-" + j + "<br>");}}}</script>
```

Versión 2:

```
<script>
    var i,j=0;
    for (i=0;i<5;i++) {
        alert("Variable i: "+i);
        for (j=0;j<5;j++) {
            if (i%2==0) {
```

```
        document.write(i + "-" + j + "<br>");
    }
}
</script>
```

EL PUNTO Y COMA

Normalmente, se suele insertar un **signo de punto y coma (;)** al **final** de cada **instrucción** de JavaScript, al igual que se hace en lenguajes de programación como C, C++ o Java.
Este signo tiene la utilidad de **separar y diferenciar cada instrucción**.
No obstante, podemos omitir dicho signo si cada instrucción de JavaScript se encuentra en una línea independiente.

Por ejemplo, las siguientes instrucciones podrían escribirse de este modo:

```
pi_egipcio = 3.16
pi_griego = 3.14
```

Sin embargo, si queremos definir los dos valores en una misma línea, es necesario utilizar al menos el primer punto y coma:

```
pi_egipcio = 3.16; pi_griego = 3.14;
```

La **omisión** del punto y coma **no es una buena práctica de programación**.

PALABRAS RESERVADAS

Las palabras reservadas son las palabras que se utilizan para construir sentencias de JavaScript y que por tanto no pueden ser utilizadas libremente:

<i>break</i>	<i>delete</i>	<i>if</i>	<i>this</i>	<i>while</i>
<i>case</i>	<i>do</i>	<i>in</i>	<i>throw</i>	<i>with</i>
<i>catch</i>	<i>else</i>	<i>instanceof</i>	<i>try</i>	
<i>continue</i>	<i>finally</i>	<i>new</i>	<i>typeof</i>	
<i>debugger</i>	<i>for</i>	<i>return</i>	<i>var</i>	
<i>default</i>	<i>function</i>	<i>switch</i>	<i>void</i>	

ETIQUETA NOSCRIPT

Algunos navegadores no disponen de soporte completo de JavaScript, otros navegadores permiten bloquearlo parcialmente e incluso algunos **usuarios bloquean completamente el uso de JavaScript** porque creen que así navegan de forma más segura. (<https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/desactivar-javascript/>)

En estos casos, es habitual que si la página web requiere JavaScript para su correcto funcionamiento, se incluya un mensaje de aviso al usuario indicándole que debería activar Javascript para disfrutar completamente de la página.

El lenguaje HTML define la etiqueta <noscript> para mostrar un mensaje al usuario cuando su navegador no puede ejecutar JavaScript.

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="utf-8">
    <title>Noscript</title>
    <script>
      ...
    </script>
  </head>
  <body>
    <noscript>
      <p>
```

La página que estás viendo requiere para su funcionamiento el uso de

```

                                JavaScript. Si lo has deshabilitado intencionadamente, por favor vuelve a
                                activarlo
                                </p>
                                </noscript>
                                </body>
                                </html>

```

La etiqueta <noscript> se debe incluir en el **interior** de la etiqueta **<body>** (normalmente se incluye al principio de <body>). El mensaje que muestra <noscript> puede incluir cualquier elemento o etiqueta HTML

ACTIVIDADES

Modifica el ejemplo Hola_mundo2 (llámale **Hola_mundo3.html** y **Hola_mundo3.js**) para que:

- a. Después del primer mensaje, muestre otro mensaje que diga “Soy el primer script”
- b. Añade algunos comentarios que expliquen el funcionamiento del código
- c. Añade en la página HTML un mensaje para los navegadores que no tengan activado el soporte JavaScript

4. TIPOS DE DATOS

Los **tres tipos de datos** primitivos de JavaScript son: **números**, **cadenas de texto** y **valores booleanos**. Además de estos tres tipos de datos, existe el tipo de datos compuesto llamado **objeto**. Este último representa **una colección de valores primitivos o de valores compuestos por otros objetos**. Esta colección de valores nos permite definir tipos de datos como por ejemplo las matrices y las funciones, las cuales explicaremos en los siguientes capítulos del libro.

NÚMEROS

Existe solo un tipo de **datos numérico**, los números se representarán a través del **formato de punto flotante de 64 bits** definido por el estándar IEEE 7542. Este formato es el llamado double en los lenguajes Java o C++. (https://es.qwe.wiki/wiki/Decimal64_floating-point_format)

CADENAS DE TEXTO

El tipo de datos que utiliza JavaScript para representar cadenas de texto, es llamado **string**. Con este tipo de datos es posible representar una secuencia de letras, dígitos, signos de puntuación o cualquier otro carácter de Unicode. La cadena de caracteres la debemos definir entre comillas dobles o comillas simples (" o ').

Para representar algunos caracteres necesitamos algunas secuencias de escape las cuales se llevan la barra invertida (\) delante:

\\	Barra invertida
\'	Comilla simple
\"	Comillas dobles
\n	Salto de línea
\t	Tabulación horizontal

Los caracteres tildados también son especiales, su codificación es \uxxxx donde xxxx es el código Unicode del carácter que queremos representar (<http://unicode-table.com/en/>)

\u00E1 = á

\n = Enter

```

<script>
    alert('Esta es mi primera p\u00E1gina \nusando secuencias de escape');
</script>

```

VALORES BOOLEANOS

El tipo de datos booleano, también conocido como **valores lógicos**, es el tipo de datos más simple debido a que admite solo dos valores: **true** o **false** (verdadero o falso).

ACTIVIDADES

Crea un fichero HTML ([Mensaje.html](#)) que permita mostrar el siguiente texto en una ventana emergente, a través de la función `alert()`:

HolaMundo!

Qué fácil es incluir ‘comillas simples’

y “comillas dobles”

5. VARIABLES

Las **variables** se pueden definir como **zonas de la memoria** de un ordenador que se identifican con un nombre y en las cuales se **almacenan ciertos datos**. Las variables nos permiten manipular y guardar datos que, en algunos casos, no sabremos su valor a priori. El desarrollo de un script conlleva generalmente dos aspectos relacionados con las variables:

- Declaración de variables
- Inicialización de variables.

DECLARACIÓN DE VARIABLES

El **primer paso** para utilizar una variable es **definirla**. En JavaScript las variables se definen a través de la palabra clave **var** seguida por el **nombre** que queramos darle a la variable:

var mi_variable_1;

var mi_variable_2;

Es posible **declarar más de una variable** en una sola línea a través de la **separación por comas**:

var mi_variable_1, mi_variable_2;

El nombre de una variable debe cumplir las siguientes normas:

- Sólo puede estar formado por **letras, números y los símbolos \$ y _**
- El **primer carácter no puede ser un número**

INICIALIZACIÓN DE VARIABLES

Podemos realizar la asignación de un valor a una variable de tres formas:

- Asignación **directa** de un valor concreto
var mi_variable_1 = 30;
- Asignación **indirecta** a través de un cálculo en el que se implican a otras variables o constantes
var mi_variable_2 = mi_variable_1 + 10;
- Asignación a través de la **solicitud del valor al usuario del programa**
var mi_variable_3 = prompt("Introduce un valor");

Si en algún fragmento del código intentamos utilizar una variable que no haya sido inicializada, JavaScript generará un error.

Podemos inicializar una variable **sin utilizar la palabra clave var**. De este modo JavaScript declara implícitamente dicha variable, pero tenemos que tener en cuenta que **esta variable será una variable global**. Esto quiere decir que su valor tendrá un ámbito global, incluso dentro de funciones locales. De este modo corremos el riesgo de modificar un valor en el cual se basa algún otro fragmento del programa.

ACTIVIDADES

Cree un nuevo fichero HTML ([Variables.html](#)) y, al igual que en las anteriores actividades, copie el siguiente código JavaScript dentro del cuerpo de la página:

<script>

var primer_saludo = "hola";

var segundo_saludo = primer_saludo;

primer_saludo = "hello";

alert(segundo_saludo);

</script>

Antes de ver el resultado, ¿cuál cree que será el valor en la ventana emergente? ¿Por qué?

6. OPERADORES

JavaScript utiliza principalmente cinco tipo de operadores:

- Operadores aritméticos.
- Operadores lógicos.
- Operadores de asignación.
- Operadores de comparación.
- Operadores condicionales.

OPERADORES ARITMÉTICOS

Los operadores aritméticos permiten realizar **cálculos elementales** entre variables numéricas.

+	Suma	Efectúa la suma entre los operandos
-	Resta	Efectúa la resta entre los operandos.
*	Multiplicación	Efectúa la multiplicación entre los operandos.
/	División	Efectúa la división entre los operandos.
%	Módulo	Resto de la división entre los operandos
++	Incremento	Permite incrementar un valor.
--	Decremento	Permite decrementar un valor.

Los operadores incremento y decremento se pueden utilizar antes o después de la variable, aunque su efecto es diferente.

Por ejemplo, el siguiente código almacena el valor 2 en ambas variables

```
variable_1 = 1;
variable_2 = ++variable_1;
```

Mientras que en este otro ejemplo la primera variable contendrá al valor 2 y la segunda 1

```
variable_1 = 1;
variable_2 = variable_1++;
```

OPERADORES LÓGICOS

Los operadores lógicos tienen la utilidad de combinar o manipular **diferentes expresiones lógicas** con el fin de **evaluar** si el resultado de esta combinación **es verdadero o falso**.

&&	Y	AND: Devuelve true solo si todos los valores son true y false en caso contrario.
	O	OR: Devuelve true en el caso en que al menos uno de los valores sea true y false en caso contrario.
!	No	Not: Invierte el valor booleano de su operando.

OPERADORES DE COMPARACIÓN

Los operadores de comparación, tal y como indica su nombre, nos permiten **comparar todo tipo de variables** con el fin de verificar sus valores. La comparación ejecutada por estos operadores se puede realizar solo sobre números y cadenas. Si utilizamos otro tipo de datos, estos se convertirán automáticamente para intentar que la comparación se pueda llevar a cabo.

<	Menor que	Devuelve true si el operando de la izquierda es menor que el operando de la derecha y false en caso contrario
<=	Menor o igual que	Devuelve true si el operando de la izquierda es menor o igual que el operando de la derecha y false en caso contrario
==	Igual	Devuelve true si los dos operandos son iguales y false en caso contrario.
>	Mayor que	Devuelve true si el operando de la izquierda es mayor que el

		operando de la derecha y false en caso contrario
<code>>=</code>	Mayor o igual que	Devuelve true si el operando de la izquierda es mayor o igual que el operando de la derecha y false en caso contrario.
<code>!=</code>	Diferente	Devuelve true si los dos operandos son diferentes y false en caso contrario.

OPERADORES CONDICIONALES

El operador condicional consta de tres partes: la primera es la expresión a evaluar, la segunda es la acción a realizar si la expresión es verdadera, y la tercera parte es la acción a realizar si la expresión es falsa.

<code>?:</code>	Condicional	Si la expresión antes del operador es verdadera, se utiliza el primer valor a la derecha de ?. En caso contrario se utiliza el segundo valor a la derecha de ?.
-----------------	-------------	---

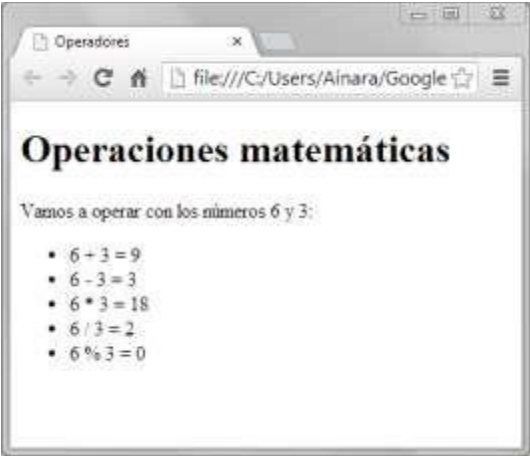
Por ejemplo, si queremos avisar al usuario que no se puede efectuar una división por cero, es posible hacerlo mediante un operador condicional:

```
<script>
  var dividendo = prompt("Introduce el dividendo: ");
  var divisor = prompt("Introduce el divisor: ");
  var resultado;
  resultado=dividendo/divisor;
  divisor != 0 ? document.write("<section>El resultado es: " + resultado + "</section>") : alert("No es posible dividir por cero");
</script>
```

ACTIVIDADES

Crea una página ([Operadores.html](#))

“Operaciones matemáticas” no tiene que formar parte del código JavaScript



7. SENTENCIAS CONDICIONALES

Podemos controlar la toma de decisiones y el posterior resultado por parte del navegador a través del uso de sentencias condicionales. Dichas sentencias permiten evaluar condiciones y ejecutar ciertas instrucciones si la condición es verdadera y ejecutar otras instrucciones diferentes si la condición es falsa.

Existen **tres tipos** de sentencias **condicionales**: la sentencia **if**, la sentencia **switch** y las sentencias en bucle **while** y **for**.

La sentencia **if** indica al navegador **si** debe ejecutar una parte de código en base al valor lógico de una expresión condicional.

La sentencia **switch** **compara** el valor de una **variable** con una serie de **valores conocidos**. Si uno de los valores conocidos coincide con el valor de la variable, se ejecuta el código asociado a dicho valor conocido.

Las sentencias en bucle **while** y **for** permiten al navegador **ejecutar un fragmento de código de forma repetida mientras la condición sea verdadera**.

SENTENCIA IF


```
if(expresión){  
    instrucciones  
}
```

La expresión puede ser una comparación o una **expresión lógica que devuelve los valores true o false**. Las instrucciones son el código que ejecutaremos si la expresión devuelve el valor true. JavaScript ignora las instrucciones en el caso en que la expresión devuelva el valor falseo

Las llaves {} no son obligatorias en el caso en que debamos ejecutar una sola instrucción si no si.

La segunda forma de utilizar la sentencia if es agregando la palabra clave else. De este modo podemos ejecutar instrucciones en el caso en que la expresión devuelva el valor falso:

```
if (expresión) {  
    instrucciones_si_true  
} else {  
    Instrucciones_si_false  
}
```

La tercera forma de utilizar la sentencia if es mediante la sentencia if-else-if. De este modo, le pedimos al navegador que evalúe una expresión, y si ésta devuelve el valor false, el navegador evaluará una nueva expresión. Si ninguna de las dos o más expresiones utilizadas devuelve el valor true, es posible utilizar un último else, donde es posible escribir el código que se ejecutará en este caso.

```
if(expresión 1) {  
    instrucciones 1  
}else if (expresión 2) {  
    instrucciones 2  
}else{  
    instrucciones 3  
}
```

En el caso en que debamos utilizar demasiadas sentencias del tipo else-if, es preferible utilizar la sentencia switch. El navegador no tendrá ningún problema en ejecutar el código, pero la lectura y mantenimiento de una sentencia compleja de varios if-else se puede convertir en una tarea difícil.

Una vez que conozcamos y dominemos el uso de la sentencia if, será posible escribir tipos de código con tomas de decisiones más complejas a través de if anidados.

SENTENCIA SWITCH

En algunos casos es necesario comparar el valor de una variable con algunos valores conocidos.

```
switch (expresión) {  
    case valor1:  
        instrucciones a ejecutar si expresión = valor1  
        break;  
    case valor2:  
        instrucciones a ejecutar si expresión = valor2  
        break;  
    case valor3:  
        instrucciones a ejecutar si expresión = valor3  
        break;  
    default:  
        instrucciones a ejecutar si expresión es diferente a los valores anteriores  
}
```

Cada caso termina con la palabra clave **break**. La utilidad de esta palabra clave es la de **detener** la

ejecución del switch en el momento en que uno de los casos resulte verdadero. De este modo no perdemos tiempo en evaluar los demás casos.

Las instrucciones que se encuentren dentro de la sentencia opcional llamada **default**, se ejecutarán solo cuando **ninguno de los valores de cada caso coincida** con el valor de la expresión.

BUCLE WHILE

A través de bucles de este tipo, el navegador ejecutará una o más instrucciones continuamente hasta que una cierta condición deje de ser verdadera.

```
while (expresión) {  
    instrucciones  
}
```

Una variación del bucle while es el denominado do-while.

```
do{  
    instrucciones  
} while (expresión)
```

De este modo nos aseguramos que la **ejecución** del bucle **se lleve a cabo al menos una vez** y solo al final, se evalúa si se debe seguir ejecutando o no.

BUCLE FOR

El bucle for permite que un navegador ejecute las instrucciones que se encuentren dentro del mismo bucle hasta que la expresión condicional devuelva el valor false.

```
for ( valor_inicial_variable; expresión condicional; incremento o decremento de la variable) {  
    instrucciones_o_cuerpo_del_bucle  
}
```

Para comprender mejor su funcionamiento, podemos utilizar un ejemplo que muestra en pantalla un valor que se incrementa en cada una de las diez iteraciones del bucle:

```
for (i=0; i<10; i++) {  
    document.write("El valor de la variable de control es: " + i + "<br>")  
}
```

ACTIVIDADES

1-Escribe un programa que pida dos números y escriba en la pantalla cual es el mayor.

([Numero_mayor.html](#))

2-Solicita por pantalla tres números y comprueba cuál de ellos es el mayor ([Numero_mayor2.html](#))



EJERCICIOS PROPUESTOS

Nombra los archivos como [Ejercicio1.html](#), [Ejercicio2.html](#)...

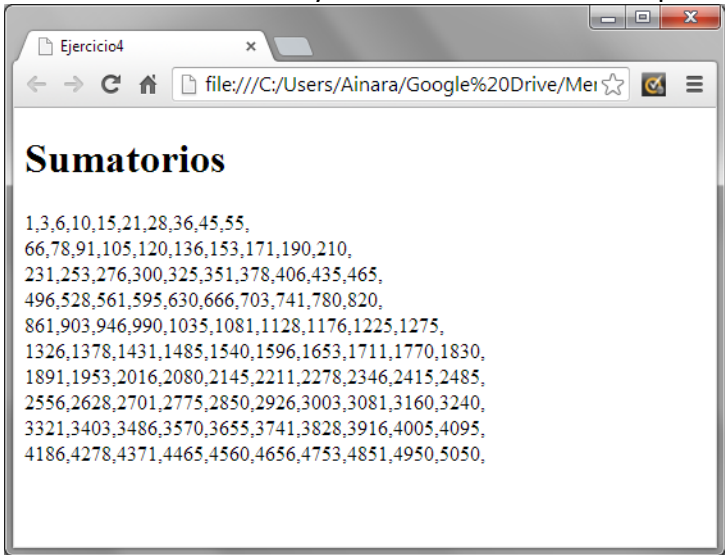
1. Solicita por pantalla un nombre y dos apellidos (con tres sentencias **prompt**) y muestra los siguientes datos:



2. Muestra los números **pares** entre 1 y 100. En **cada fila deben aparecer 5 números**. Usa un bucle **FOR** para resolver el ejercicio.

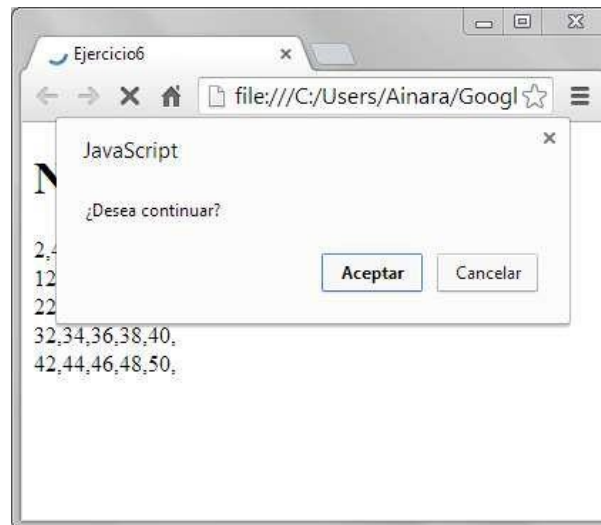


3. Resuelve el ejercicio 2 haciendo uso de un bucle **while**.
4. Muestra el sumatorio de cada valor entre 1 y 100. En cada fila deben aparecer 10 números.



5. Resuelve el ejercicio 4 haciendo uso de un bucle **while**.

6. Haciendo uso de la función **confirm** (https://www.w3schools.com/js/js_popup.asp) de JavaScript y de un bucle **do ... while** muestra los números pares de 25 en 25 hasta que el usuario decida **terminar**.



Confirm: función de JavaScript que abre una ventana con dos botones Aceptar y Cancelar. Si se pulsa el botón Aceptar la función devuelve el valor True y si se pulsa el botón Cancelar devuelve el valor False

7. Muestra los sumatorios impares inferiores a 1000 de los números comprendidos entre 1 y 100. En cada fila se mostrarán 4 sumatorios como máximo. Usa la sentencia **break** para finalizar el bucle principal cuando se encuentre el primer sumatorio con valor superior a 1000.



8. El factorial de un número entero n es una operación matemática ($n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$). Utilizando la estructura for, crear un script que calcule el factorial de un número solicitado por pantalla y lo muestre mediante la función **alert()**. (Usar la función **parseInt** https://www.w3schools.com/jsref/jsref_parseint.asp)

Ejemplos:

$3! = 3 \cdot 2 \cdot 1 = 6$

$4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24$

$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$